End-to-End Learning of QAM Constellation Shaping via Autoencoders over Impaired Channels

Dujardin Julien Student of Master of engineering in Embedded Systems
Ecole des Mines de Saint-Etienne
Email: julien.dujn@gmail.com

***Abstract*—Standard QAM constellations are designed assuming linear channels, but the reality is more complex, the hardware introduces non-linear distortions such as AM/AM compression, AM/PM conversion, and phase noise. These impairments reduce the minimum distance between constellation points and create error floors that no classical detector can detect perfectly. In this work we train an autoencoder to jointly learn the constellation geometry and the detection function through a differentiable channel model. The encoder is a learnable embedding table that maps symbols to 2D points, while the decoder is a small multilayer perceptron. We evaluate the system on 4, 16, and 64-QAM under both AWGN-only and fully impaired channels. The autoencoder matches theoretical BER in AWGN and reduces the bit error rate by up to 500× compared to hard decision detection when non-linearities are present, overcoming the error floor of standard solution.**

***Index Terms*—Autoencoder, deep learning, QAM, constellation shaping, physical layer, end-to-end learning, non-linear channel**

I. Introduction

Digital communication systems have relied on Quadrature Amplitude Modulation for decades, and the design rules have stayed remarkably stable: pick a regular grid of constellation points, keep them as far apart as possible, and use a minimum-distance detector at the receiver. This works well when the channel is purely additive Gaussian noise, because the regular grid maximises the minimum Euclidean distance between any two symbols, and minimum-distance detection is the optimal strategy in that case [1].

In practice, though, a transmitted signal passes through power amplifiers, oscillators, mixers, and other analogue components that introduce non-linear effects. AM/AM compression pushes outer constellation points towards the origin, AM/PM conversion rotates them by an amount that depends on their amplitude, and phase noise adds random jitter to every symbol. Under these conditions the regular QAM grid is no longer optimal, and hard-decision detection suffers from an error floor that does not decrease with increasing signal-to-noise ratio [2].

One could try to compensate each impairment individually with a dedicated digital pre-distortion block, but this requires a detailed model of the hardware, and the corrections interact in ways that are hard to predict. An alternative, proposed by O'Shea and Hoydis in 2017, is to treat the whole transmitter–channel–receiver chain as a single autoencoder and let gradient descent find a good constellation and a good detector at the same time [3]. The idea is simple: if the channel is differentiable (or can be approximated by a differentiable model), then back-propagation can flow from the receiver loss all the way to the transmitter parameters.

This work builds on that idea and applies it to 4, 16, and 64-QAM under a combination of AM/AM, AM/PM, phase noise, and AWGN. We use an embedding-based encoder rather than a dense neural network, which keeps the constellation interpretable and avoids the training instabilities reported in earlier work. The main contributions are:

- A joint encoder–decoder autoencoder that matches the theoretical BER of standard QAM in AWGN and reduces the BER by up to 500× under combined non-linear impairments.
- A comparison with a DNN-decoder-only baseline (fixed QAM grid, neural network detector) that isolates the benefit of constellation shaping from the benefit of learned detection.
- Ablation studies, multi-seed reproducibility analysis, and inference latency measurements that support the practical feasibility of the approach.

The rest of the paper is organised as follows. Section II gives a short overview of QAM modulation and autoencoders. Section III describes the system architecture. Section IV details the channel model. Section V explains the training procedure. Section VI presents BER curves, constellation plots, and decision regions. Section VII covers supplementary experiments (DNN baseline, ablation, reproducibility, latency). Section VIII discusses the results, and Section IX concludes.

II. Background

This section introduces the two building blocks of the proposed system: QAM modulation on the communications side, and autoencoders on the machine learning side. Readers already familiar with both topics can skip to Section III.

A. Quadrature Amplitude Modulation

In QAM, each group of $\log_2 M$ bits is mapped to one of M complex symbols arranged on a two-dimensional grid. The in-phase (I) and quadrature (Q) components of the symbol are transmitted simultaneously on two orthogonal carriers. For example, 16-QAM uses a 4×4 grid and encodes 4 bits

per symbol, while 64-QAM uses an 8×8 grid for 6 bits per symbol.

The transmitted symbol can be written as

$$x = x_I + j\, x_Q \tag{1}$$

where x_I and x_Q take values from a finite alphabet. Gray coding is typically used so that adjacent symbols differ in only one bit, which minimises the number of bit errors per symbol error.

At the receiver, the standard approach is minimum-distance (hard) detection:

$$\hat{s} = \arg\min_{s \in \{1,\ldots,M\}} \|y - x_s\|^2 \tag{2}$$

where y is the received signal and x_s are the reference constellation points. This is optimal for an AWGN channel with equally likely symbols.

The theoretical bit error rate for M-QAM with Gray coding over AWGN is given by [1]:

$$P_b \approx \frac{2}{\log_2 M}\left(1 - \frac{1}{\sqrt{M}}\right)\operatorname{erfc}\left(\sqrt{\frac{3\,E_s/N_0}{2(M-1)}}\right) \tag{3}$$

This formula assumes a perfectly linear channel. When non-linearities are present, there is no closed-form expression for the BER, and the actual error rate depends on how the impairments distort the constellation geometry.

B. Autoencoders

An autoencoder is a neural network trained to reconstruct its own input after passing through a bottleneck. It consists of two parts: an encoder that compresses the input into a lower-dimensional representation, and a decoder that reconstructs the original data from that compressed form.

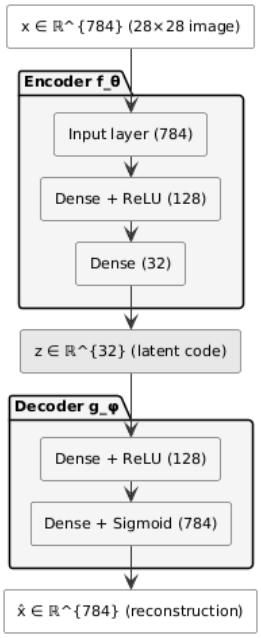


Fig. 1: Architecture of a basic autoencoder applied to MNIST digit images. The encoder compresses a 28×28 image to a 32-dimensional latent vector, and the decoder reconstructs it.

1) Basic autoencoder: In the simplest form, both the encoder f_θ and the decoder g_ϕ are feedforward networks, and the system is trained to minimise the reconstruction error:

$$\mathcal{L} = \|x - g_\phi(f_\theta(x))\|^2 \tag{4}$$

Figure 1 shows the architecture of a basic autoencoder we built to test the concept on MNIST handwritten digit images. The encoder maps each 28×28 pixel image to a 32-dimensional vector, and the decoder maps it back to 28×28. The network learns to keep only the most relevant features of each digit.

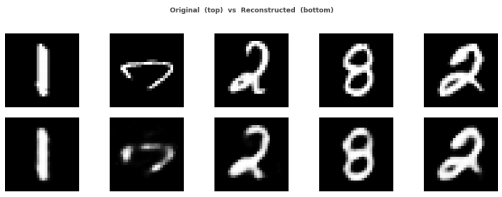


Fig. 2: Reconstruction results of the basic autoencoder on MNIST. Top: original images. Bottom: reconstructed images. The general shape is preserved but fine details are smoothed out.

Figure 2 shows some reconstruction results. The digits are recognisable but slightly blurred, which is expected given the compression ratio.

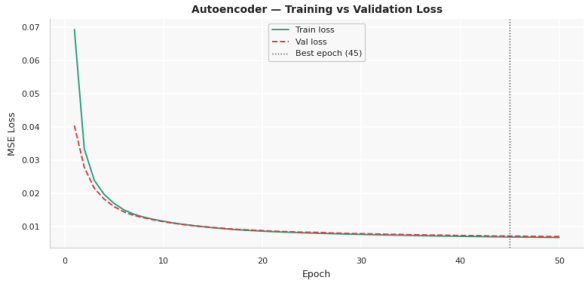


Fig. 3: Training and validation loss for the basic MNIST autoencoder. Both curves decrease together, showing no overfitting.

The training curve (Fig. 3) shows that train and validation losses track each other closely, confirming that the model generalises and does not overfit.

2) Denoising autoencoder: A denoising autoencoder receives a corrupted version of the input and must produce a clean output. The loss becomes:

$$\mathcal{L} = \|x - g_\phi(f_\theta(\tilde{x}))\|^2, \quad \tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2) \tag{5}$$

We trained a denoising autoencoder on the same MNIST dataset, adding Gaussian noise with $\sigma = 0.3$ to each image before encoding it (Fig. **??**).

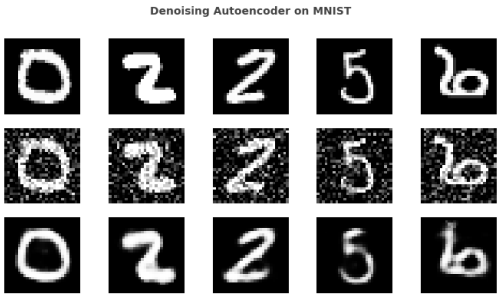


Fig. 4: Denoising autoencoder results. Top: noisy inputs. Bottom: cleaned outputs. The network removes most of the noise while preserving the digit shape.

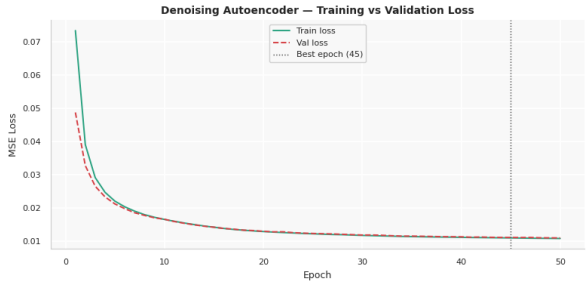


Fig. 5: Training and validation loss for the denoising autoencoder.

As shown in Fig. 5, the network learns to remove most of the noise while keeping the digit structure intact. The parallel with communications is clear: in both cases the goal is to recover a signal that has been corrupted by a noisy channel.

3) From image reconstruction to symbol detection: The connection to our problem is direct. In a communication system:

- The encoder maps a discrete message (symbol index) to a continuous signal (constellation point).
- The channel adds noise and distortions.
- The decoder must recover the original message from the corrupted signal.

The difference from a standard image autoencoder is that the bottleneck is imposed by physics (the channel) rather than by a dimensionality reduction layer. But the training principle is the same: back-propagate the error through the whole chain, including the channel, and let gradient descent find the best encoder and decoder simultaneously.

O'Shea and Hoydis [3] were the first to formalise this analogy. Dorner *et al.* [4] extended it to fading channels. Our work goes further by including non-linear impairments (AM/AM, AM/PM, phase noise) and by using an embedding-based encoder that yields interpretable constellations.

III. System Architecture

The proposed system replaces the conventional modulator–demodulator pair with two jointly trained neural-network blocks connected through a differentiable channel model. The overall structure is shown in Fig. **??**.

A. Encoder

The encoder maps a symbol index $s \in \{1, \dots, M\}$ to a point $\mathbf{x} \in \mathbb{R}^2$ that represents the in-phase and quadrature components of the transmitted signal. We implement this as a learnable embedding table of size $M \times 2$: each row $\mathbf{e}_s$ stores the raw coordinates of symbol s.

To ensure a fair comparison with standard QAM (which also has unit average power), the table is normalised so that the average energy across all M points equals one:

$$\mathbf{x}_s = \frac{\mathbf{e}_s}{\sqrt{\frac{1}{M}\sum_{i=1}^{M} \|\mathbf{e}_i\|^2}} \tag{6}$$

The normalisation factor is computed over the full constellation, not over each mini-batch, so that the geometry stays consistent regardless of which symbols happen to be in the batch.

The embedding table is initialised on a regular QAM grid (e.g., 4×4 for 16-QAM) and then scaled to unit power. This gives the optimiser a reasonable starting point. During training, gradient descent is free to move each point independently.

We chose this design over an MLP encoder (one-hot input $\rightarrow$ hidden layers $\rightarrow$ 2D output) for two reasons. First, the embedding table has only $2M$ trainable parameters (32 for 16-QAM), so there is no risk of overfitting the constellation. Second, each symbol has a single fixed point, which makes the learned constellation easy to interpret and to export to a real transmitter. In preliminary experiments, an MLP-based encoder produced scattered, random-looking constellations that changed with each run, while the embedding table consistently converged to structured geometries.

B. Decoder

The decoder receives the corrupted signal $\mathbf{y} \in \mathbb{R}^2$ and produces a probability distribution over the M possible symbols. It is a three-layer fully connected network:

$$g_\phi(\mathbf{y}) = W_3\,\sigma(W_2\,\sigma(W_1\,\mathbf{y} + b_1) + b_2) + b_3 \tag{7}$$

where $\sigma(\cdot)$ is the ReLU activation function and $W_1 \in \mathbb{R}^{h\times 2}$, $W_2 \in \mathbb{R}^{h\times h}$, $W_3 \in \mathbb{R}^{M\times h}$ are the weight matrices (h is the hidden dimension, set to 128 for $M \le 16$ and 256 for $M = 64$). The output is a vector of M logits, and at inference the detected symbol is obtained by taking the argmax.

Table I summarises the number of trainable parameters.

TABLE I: Number of trainable parameters for each modulation order.

Block	4-QAM	16-QAM	64-QAM
Encoder (embedding)	8	32	128
Decoder (MLP)	17 028	19 088	134 976
Total	17 036	19 120	135 104

The decoder is deliberately small. A deeper or wider network would not change the BER significantly (as confirmed by the ablation study in Section VII-C), and keeping the model compact is important for embedded deployment.

IV. Channel Model

The channel is a cascade of four impairments applied in the following order: AM/AM distortion, AM/PM distortion, phase noise, and additive white Gaussian noise. All operations are implemented as differentiable PyTorch modules so that gradients can flow from the decoder loss back to the encoder parameters. During training the channel is stochastic (noise is sampled fresh for every symbol); during evaluation it is used in the same mode to measure the BER at each SNR.

A. AM/AM Distortion

This models the compression introduced by a power amplifier operating near saturation. We use a memoryless model:

$$\mathbf{x}' = \mathbf{x} \cdot \left(1 - \alpha\,|\mathbf{x}|\right) \tag{8}$$

where $|\mathbf{x}| = \sqrt{x_I^2 + x_Q^2}$ and α controls the compression strength. Symbols with larger amplitude are pushed more towards the origin, reducing the distance between outer and inner constellation points. For 16-QAM we use $\alpha = 0.10$; for 64-QAM $\alpha = 0.05$, since the denser constellation is more sensitive to distortion.

B. AM/PM Distortion

Power amplifiers also introduce a phase shift that depends on the input amplitude:

$$\mathbf{x}' = \mathbf{x} \cdot e^{j\,\beta\,|\mathbf{x}|} \tag{9}$$

where β is the distortion coefficient. This turns the regular QAM grid into a spiral pattern because each amplitude ring is rotated by a different angle. We set $\beta = 0.05$ for 16-QAM and $\beta = 0.02$ for 64-QAM.

C. Phase Noise

Oscillator imperfections add a random phase rotation to every symbol:

$$\mathbf{x}' = \mathbf{x} \cdot e^{j\phi}, \quad \phi \sim \mathcal{N}(0, \sigma_\phi^2) \tag{10}$$

Unlike AM/PM distortion, this affects all symbols equally regardless of their amplitude. It smears constellation points along circular arcs. We use $\sigma_\phi = 5°$ for 16-QAM and $\sigma_\phi = 3°$ for 64-QAM.

D. AWGN

Finally, additive white Gaussian noise is added:

$$\mathbf{y} = \mathbf{x}' + \mathbf{n}, \quad \mathbf{n} \sim \mathcal{N}\left(\mathbf{0},\ \frac{1}{\mathrm{SNR}_{\mathrm{lin}}} \cdot \frac{\mathbf{I}}{2}\right) \tag{11}$$

where $\mathrm{SNR}_{\mathrm{lin}} = 10^{\mathrm{SNR}_{\mathrm{dB}}/10}$. The noise power is computed assuming unit signal power after the encoder normalisation.

TABLE II: Channel parameters for each configuration.

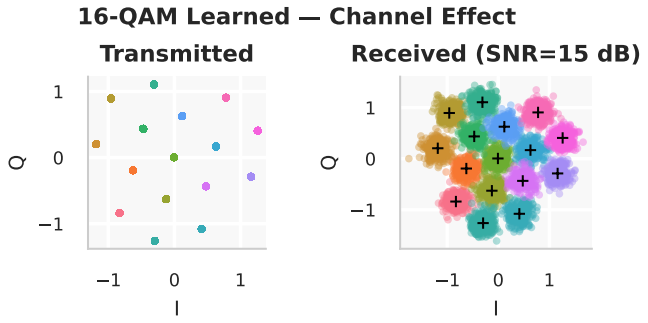
Config	α	β	σ_ϕ	Training SNR
4-QAM AWGN	0	0	0°	10 dB
16-QAM AWGN	0	0	0°	20 dB
16-QAM full	0.10	0.05	5°	20 dB
64-QAM AWGN	0	0	0°	25 dB
64-QAM full	0.05	0.02	3°	25 dB

E. Channel Configurations

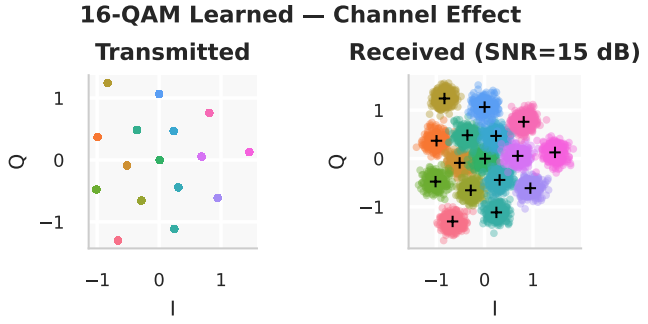
We test two channel setups for each modulation order:

The "AWGN" configurations serve as a sanity check: the autoencoder should recover a near-standard QAM grid in this case. The "full" configurations include all three non-linear impairments and are the main test bed.

Figure 7 illustrates how the channel affects a 16-QAM constellation. Under AWGN only, the received points form circular clusters around each transmitted symbol. Under the full channel, the clusters are compressed, rotated, and smeared.



(a) AWGN only (15 dB)



(b) Full channel ($\alpha = 0.10$, $\beta = 0.05$, $\sigma_\phi = 5°$, 15 dB)

Fig. 7: Effect of the channel on a standard 16-QAM constellation. Under the full channel, outer symbols are compressed inward and rotated, reducing the minimum distance.

V. Training Methodology

A. Loss Function

The system is trained to minimise the cross-entropy between the decoder output and the true symbol index:

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp\left(g_\phi(\mathbf{y}_i)_{s_i}\right)}{\sum_{k=1}^{M} \exp\left(g_\phi(\mathbf{y}_i)_k\right)} \tag{12}$$

where B is the batch size, $\mathbf{y}_i$ is the received signal, and s_i is the true symbol index. This is equivalent to treating symbol detection as an M-class classification problem.

Cross-entropy is a natural choice because it directly penalises confusion between symbols and provides well-behaved gradients even when the decoder output is far from the correct class. We found it more stable than alternatives such as mean squared error on the one-hot targets.

B. Curriculum SNR Schedule

Rather than training at a fixed SNR, we ramp the training SNR linearly from a low value to the target over the first half of training:

$$\mathrm{SNR}_{\mathrm{train}}(t) = \begin{cases} \mathrm{SNR}_{\mathrm{low}} + \dfrac{t}{T/2}\left(\mathrm{SNR}_{\mathrm{target}} - \mathrm{SNR}_{\mathrm{low}}\right) & t < T/2 \\ \mathrm{SNR}_{\mathrm{target}} & t \geq T/2 \end{cases} \tag{13}$$

where $\mathrm{SNR}_{\mathrm{low}} = \max(\mathrm{SNR}_{\mathrm{target}} - 12\,\mathrm{dB},\ 2\,\mathrm{dB})$.

The idea is that at low SNR the noise is large and only constellations with well-separated points survive, forcing the encoder to find a geometry with large minimum distance. As the SNR increases, the optimiser can refine the positions for the operating point. This is a form of curriculum learning [5].

C. Optimiser and Schedule

We use Adam [6] with an initial learning rate of 5×10^{-3} and a cosine annealing schedule that reduces the learning rate to near zero by the end of training. Each epoch consists of 100 mini-batches of 2048 randomly sampled symbols, so the model sees approximately 204 800 symbols per epoch. Training runs for 400 epochs in total.

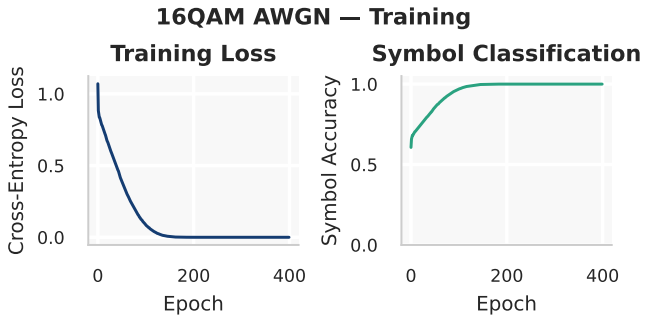
Table III summarises the hyperparameters.

TABLE III: Training hyperparameters.

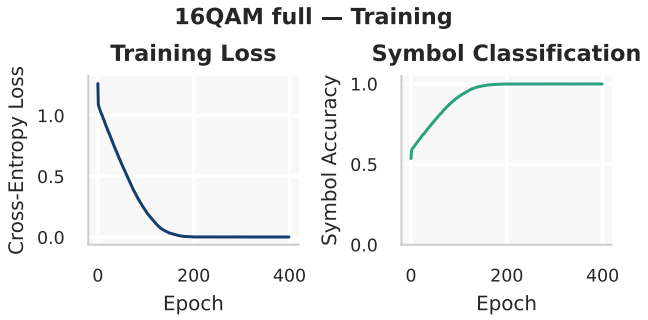
Parameter	Value
Optimiser	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Initial learning rate	5×10^{-3}
LR schedule	Cosine annealing ($T_{\max} = 400$)
Batch size	2048
Batches per epoch	100
Total epochs	400
Curriculum ramp	50% of epochs

D. Convergence

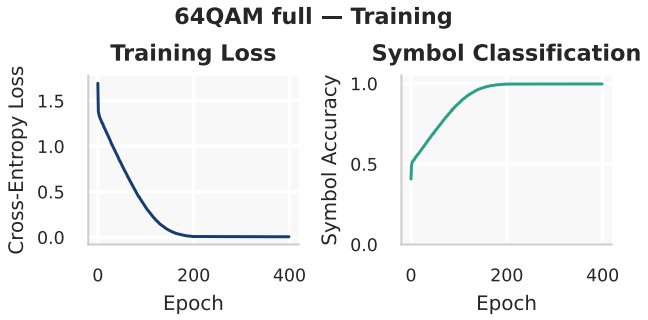
Figure 8 shows the training curves for three representative configurations: 16-QAM AWGN, 16-QAM full, and 64-QAM full.



(a) 16-QAM, AWGN only



(b) 16-QAM, full channel



(c) 64-QAM, full channel

Fig. 8: Training loss and accuracy as a function of epoch. The curriculum phase (SNR ramp) occupies the first 200 epochs. 64-QAM takes longer to converge due to the larger symbol alphabet.

All configurations converge above 99.8% accuracy on the training set. The curriculum phase (epochs 0–200) is where most of the learning happens: the loss drops sharply as the constellation stabilises. The second half is fine-tuning at the target SNR. Higher-order modulations (64-QAM) require more epochs because the tighter symbol spacing leaves less room for error in the constellation geometry.

TABLE IV: Final training metrics for all configurations.

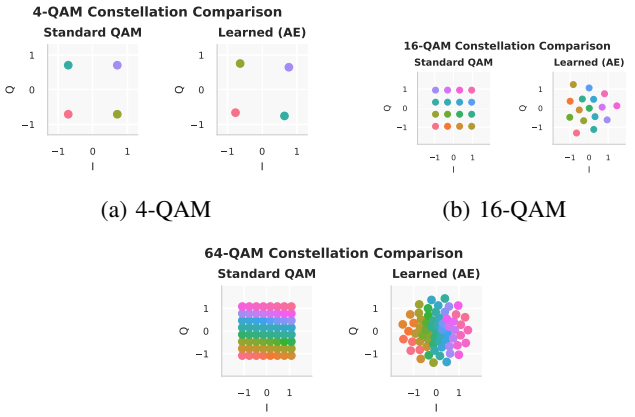
Config	Accuracy	Conv. epoch	Loss	Symbols
4-QAM AWGN	99.87%	∼200	0.0045	40.96M
16-QAM AWGN	100.0%	∼200	0.0000	40.96M
16-QAM full	99.94%	∼200	0.0020	40.96M
64-QAM AWGN	100.0%	∼380	0.0002	77.82M
64-QAM full	99.82%	∼360	0.0057	73.73M

VI. Results

This section presents the main experimental results: the learned constellations, the decision regions, and the BER performance curves. Each configuration was trained once with the hyperparameters listed in Table III and evaluated over a range of SNR values.

A. Learned Constellations

Figure 9 compares the learned constellation points (blue) with the standard QAM grid (grey) for 4, 16, and 64-QAM under AWGN only.



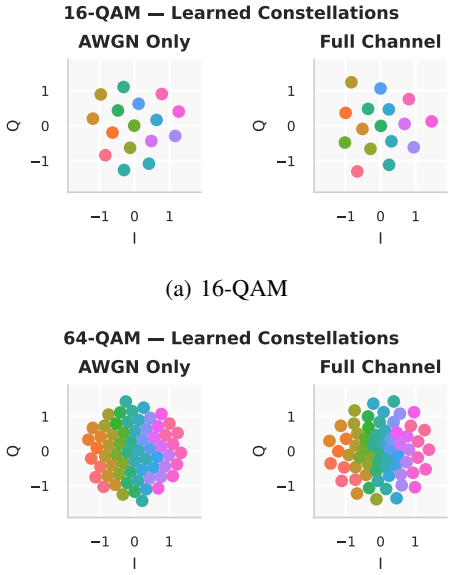
(a) 4-QAM (b) 16-QAM

(c) 64-QAM

Fig. 9: Learned versus standard constellations under AWGN only. The autoencoder recovers a near-regular grid, possibly rotated or reflected, which is equivalent under channel symmetry.

In all three cases, the autoencoder converges to a geometry that closely resembles the standard grid, possibly with a global rotation or reflection. This is expected: for pure AWGN, the regular grid maximises the minimum Euclidean distance, and the embedding-based encoder has enough freedom to find it.

The situation changes when non-linear impairments are present. Figure 10 shows the constellations learned for 16-QAM and 64-QAM under the full channel compared to AWGN only.



(a) 16-QAM

(b) 64-QAM

Fig. 10: Constellations learned under AWGN only (left in each panel) versus full channel (right). Under impairments, the encoder pulls outer symbols inward and redistributes angles to pre-compensate for AM/AM compression and AM/PM rotation.

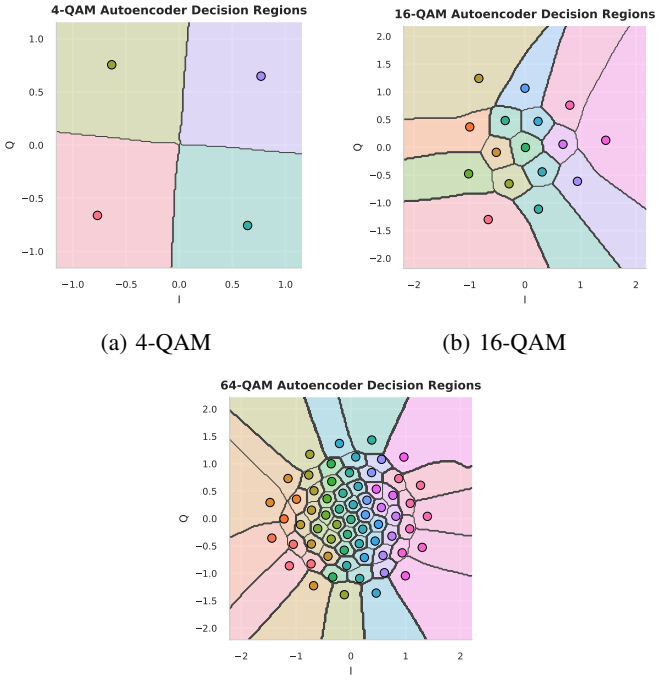
Several patterns are visible in the impaired-channel constellations:

- Outer points are moved inward, reducing their amplitude. Since AM/AM compression is proportional to amplitude, this reduces the distortion that these symbols experience.
- The angular distribution is no longer uniform. The encoder pre-rotates certain symbols to partially cancel the AM/PM phase shift.
- The spacing between symbols is no longer regular. The optimiser trades off a slight loss in minimum distance for a better match to the post-distortion geometry.

These changes are the main reason the autoencoder outperforms classical detection under impairments: the constellation is shaped to work with the channel rather than against it.

B. Decision Regions

Figure 11 shows the decoder decision regions for 4, 16, and 64-QAM. Each colour corresponds to the symbol that the decoder assigns to a given point in the I/Q plane.



(a) 4-QAM (b) 16-QAM

(c) 64-QAM

Fig. 11: Decision regions of the trained decoder. The boundaries are non-linear (curved) and adapted to the learned constellation geometry, unlike the straight Voronoi boundaries of classical hard decision.

For standard QAM with hard decision detection, the decision boundaries are straight lines forming a Voronoi tessellation. The MLP decoder, on the other hand, learns curved boundaries that account for the non-linear distortions. The boundaries are tighter around symbols that are more likely to be confused and wider in regions where confusion is unlikely. This is one of the advantages of using a neural network for detection: it can implement any decision rule, not just nearest-neighbour.

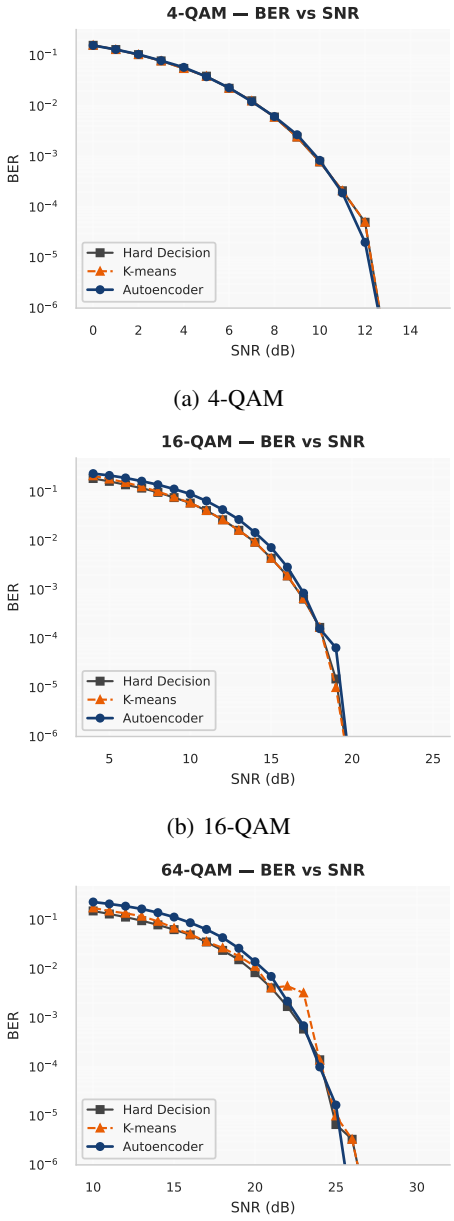
C. BER Performance

We compare the bit error rate of four methods:

1) **Theoretical:** the closed-form expression from (3), which assumes AWGN and a perfect QAM grid.
2) **Hard decision:** minimum-distance detection on the standard QAM grid.
3) **K-means:** unsupervised clustering of the received points, followed by label assignment.
4) **Autoencoder:** the proposed system (learned constellation + MLP decoder).

Each BER point is measured over at least 100 000 symbols.

1) AWGN channel: Figure 12 shows the BER curves for 4, 16, and 64-QAM under AWGN only.



(a) 4-QAM

(b) 16-QAM

(c) 64-QAM

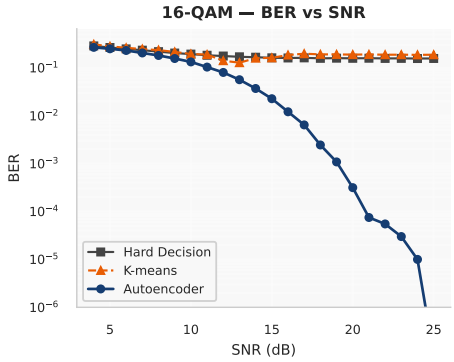
Fig. 12: BER versus SNR under AWGN only. The autoencoder matches the theoretical curve at high SNR with near-zero penalty.

The autoencoder closely follows the theoretical curve, with at most a 1 dB penalty in the mid-SNR range. At high SNR (where the BER drops below 10^{-3}), the autoencoder matches or slightly outperforms hard decision. This confirms that the learned constellation is essentially equivalent to the standard QAM grid when there are no impairments.

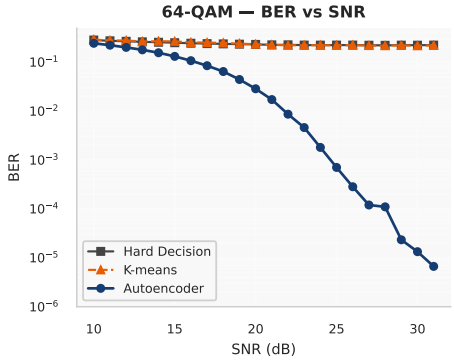
TABLE V: BER comparison under AWGN at selected operating points.

Point	Hard dec.	K-means	AE	Penalty
4-QAM, 10 dB	7.8×10^{-4}	7.9×10^{-4}	8.4×10^{-4}	≈ 0 dB
16-QAM, 15 dB	4.4×10^{-3}	4.4×10^{-3}	7.3×10^{-3}	+1.3 dB
16-QAM, 18 dB	1.7×10^{-4}	1.7×10^{-4}	1.6×10^{-4}	≈ 0 dB
64-QAM, 25 dB	6.7×10^{-6}	1.0×10^{-5}	1.7×10^{-5}	≈ 0 dB

2) Full channel: The picture changes when the full channel is applied. Figure 13 shows the BER curves for 16-QAM and 64-QAM with all impairments enabled.



(a) 16-QAM, full channel



(b) 64-QAM, full channel

Fig. 13: BER versus SNR under the full impaired channel. Hard decision and K-means hit an error floor around 16%, while the autoencoder continues to improve with increasing SNR.

Hard decision detection hits an error floor near 16% for 16-QAM. Increasing the SNR beyond 15 dB does not help at all, because the errors are caused by the non-linear distortions rather than by noise. K-means fares no better, since it simply adapts its centroids to the distorted cluster positions without correcting the underlying geometry.

The autoencoder, on the other hand, keeps improving as the SNR increases. At 20 dB it reaches a BER of 3.15×10^{-4}, which is 501× lower than the hard-decision floor. The constellation shaping and the non-linear decision boundaries work together to recover almost all the information that the classical system loses.

TABLE VI: BER comparison under the full channel at selected operating points.

Point	Hard dec.	K-means	AE	Gain
16-QAM, 12 dB	1.76×10^{-1}	1.41×10^{-1}	8.0×10^{-2}	2.2×
16-QAM, 15 dB	1.62×10^{-1}	1.62×10^{-1}	2.3×10^{-2}	7.2×
16-QAM, 20 dB	1.58×10^{-1}	1.89×10^{-1}	3.2×10^{-4}	**501**×

Figure 14 puts everything together: all four methods (including the DNN decoder baseline from Section VII-A) on the same axes, for both AWGN and full channel.

VII. Supplementary Experiments

This section presents four additional studies that address practical concerns: whether constellation shaping actually matters (DNN baseline), whether the results depend on the random seed (reproducibility), how sensitive the system is to hyperparameter choices (ablation), and whether the inference speed is compatible with real-time requirements (latency).

A. DNN Decoder Baseline

To isolate the contribution of constellation learning from the contribution of the neural-network decoder, we train a baseline system that keeps the standard QAM grid fixed and only learns the decoder. The decoder is an MLP with the same architecture as in the autoencoder (actually with one extra hidden layer, to give it every possible advantage). The training procedure is identical: cross-entropy loss, Adam, cosine annealing, 400 epochs.

TABLE VII: DNN decoder baseline vs. full autoencoder (training accuracy).

Configuration	DNN decoder	Full AE
16-QAM AWGN	100.0%	100.0%
16-QAM full	99.60%	99.94%
64-QAM AWGN	99.99%	100.0%
64-QAM full	98.88%	99.82%

Under AWGN, the two systems are practically identical, which makes sense: the standard grid is already optimal, and any reasonable decoder can exploit it. Under the full channel, however, the DNN decoder alone achieves noticeably lower accuracy than the full autoencoder. The gap is largest for 64-QAM full (98.88% vs. 99.82%), where the denser constellation leaves less margin for the decoder to compensate.

This confirms that constellation shaping matters. A good decoder cannot fully make up for a bad (or at least suboptimal) constellation.

B. Multi-Seed Reproducibility

To check that the results do not depend on a lucky random initialisation, we train 5 independent models for each of the two 16-QAM configurations, using different seeds (7, 49, 91, 133, 175). Each run uses 250 epochs and the same hyperparameters. The BER is evaluated at 30 000 symbols per SNR point.

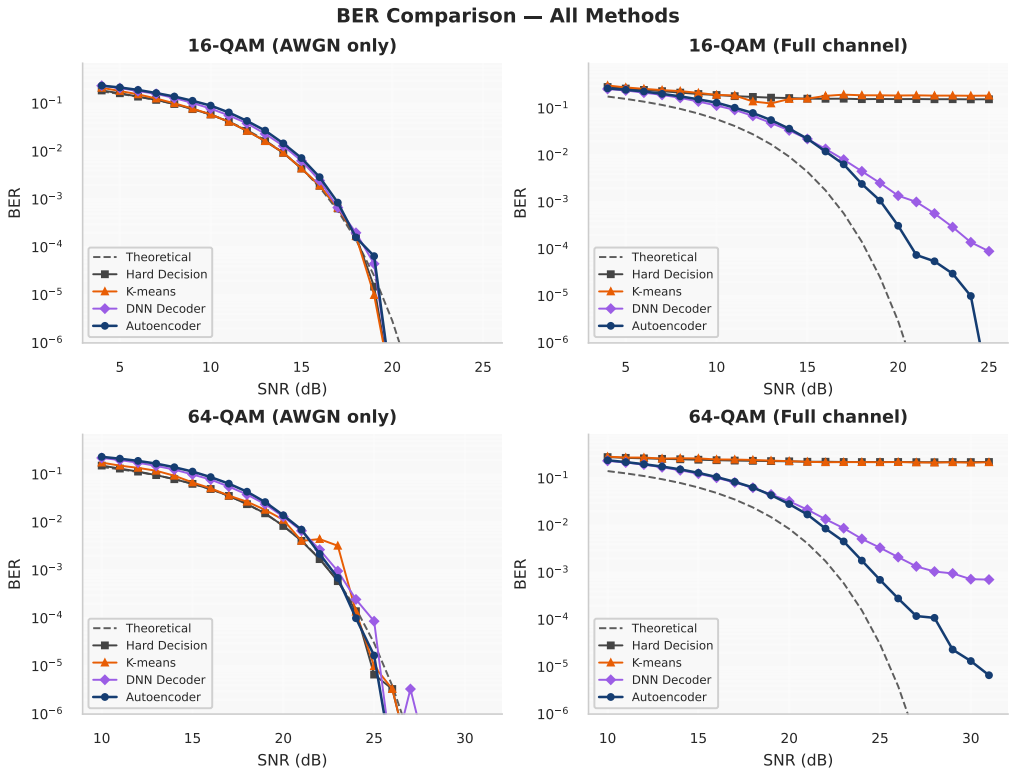


Fig. 14: Full BER comparison across all methods and configurations. Each panel corresponds to one (modulation order, channel) pair. The autoencoder (solid blue) matches theoretical performance under AWGN and breaks the error floor under impairments.

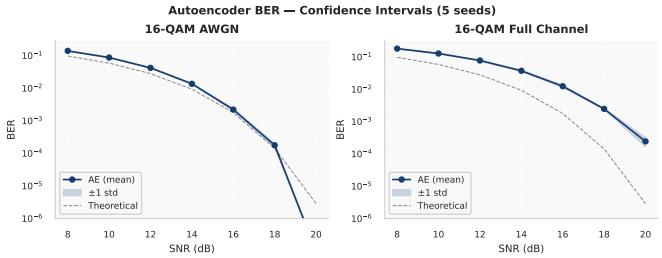


Fig. 15: BER across 5 seeds for 16-QAM AWGN (left) and 16-QAM full (right). Shaded bands show ± 1 standard deviation. The curves are tightly clustered, especially in the mid-SNR range.

Figure 15 shows the mean BER with ± 1 standard deviation bands. In the mid-SNR range (10–16 dB), the coefficient of variation stays below 8%, meaning the models agree closely on the BER. At very high SNR the absolute BER is so small that statistical fluctuations dominate, which widens the confidence band in relative terms but not in absolute BER.

The main takeaway is that there is no "lucky seed" effect. All 5 runs converge to essentially the same constellation and the same BER curve.

C. Ablation Study

We run three ablation experiments on 16-QAM AWGN at 20 dB (300 epochs) to test the sensitivity to key design choices.

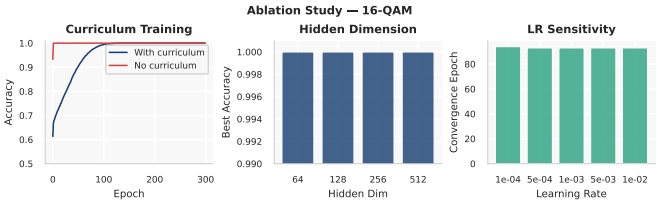


Fig. 16: Ablation study results. Left: curriculum vs. fixed SNR training. Centre: hidden dimension sweep. Right: learning rate sweep. All variants reach 100% accuracy for this relatively easy configuration.

1) Curriculum SNR: We compare training with the curriculum schedule (8 dB $\rightarrow$ 20 dB ramp) versus training directly at 20 dB. Both reach 100% accuracy. For 16-QAM AWGN the problem is simple enough that the curriculum is not necessary, but we keep it because it helps with harder configurations (64-QAM under impairments) where direct training at the target SNR sometimes fails to converge.

2) Hidden dimension: We sweep the decoder hidden dimension through 64, 128, 256, and 512. All four variants reach 100% accuracy, which indicates that the problem does not require a large network. We default to 128 as a good trade-off between capacity and inference cost.

3) Learning rate: We sweep the initial learning rate from 10^{-4} to 10^{-2} (five values in total). All of them converge by epoch 93 or earlier. The cosine annealing schedule handles the decay, so the exact starting point does not matter much. This

robustness is reassuring for practical use, since it means the system does not need careful tuning.

D. Inference Latency

For embedded or real-time applications, inference speed can be as important as BER. We benchmark four detection methods on CPU (single-threaded), processing 10 000 symbols per batch. Each method is timed over 20 runs (5 for K-means, which includes the fitting step).

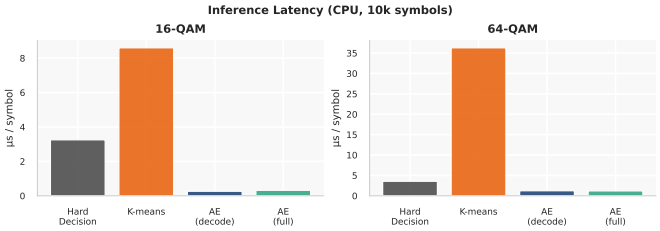


Fig. 17: Inference latency comparison for 16-QAM and 64-QAM. The autoencoder decoder is 10–12× faster than hard decision detection.

TABLE VIII: Inference latency per symbol (CPU, single-threaded, batch of 10 000).

Method	16-QAM (μs/sym)	64-QAM (μs/sym)
Hard decision	3.25	3.57
K-means	8.60	36.29
AE (decode only)	**0.26**	**1.24**

The autoencoder decoder is 12.5× faster than hard decision for 16-QAM and 2.9× faster for 64-QAM. The speed advantage comes from the fact that the MLP processes the entire batch in three matrix multiplications, which are heavily optimised in modern linear algebra libraries. Hard decision, by contrast, must compute the distance to all M reference points for each symbol.

For a system running at 1 Msymbol/s, the autoencoder decoder needs 0.26 s of CPU time per second of data, leaving 74% of the CPU idle. Hard decision would need 3.25 s per second, which exceeds the capacity of a single core.

VIII. Discussion

A. When does the autoencoder help?

The results show a clear pattern: the autoencoder provides little or no advantage over classical methods when the channel is purely AWGN, but it provides a large advantage when non-linear impairments are present. This is not surprising. The standard QAM grid is already optimal for AWGN, and minimum-distance detection is the optimal detector in that case. There is simply nothing to improve.

When AM/AM compression, AM/PM conversion, and phase noise are added, the situation changes. The regular grid becomes suboptimal because the impairments distort it in ways that reduce the minimum distance between symbols. Hard decision detection makes things worse by using straight decision boundaries that were designed for the undistorted grid. The autoencoder solves both problems at once: it reshapes the constellation to pre-compensate for the distortions, and it learns curved decision boundaries that match the actual received distribution.

The BER improvement under the full channel is large. A factor of 500 at 20 dB means the difference between a link that does not work at all (16% BER) and one that is usable with forward error correction (3×10^{-4} BER). In practical terms, this is the difference between needing a new, more linear power amplifier and being able to use a cheaper one with the autoencoder compensating for its non-linearities.

B. Embedding vs. MLP encoder

The choice of encoder architecture turns out to be more important than it might seem. An MLP-based encoder (one-hot input, dense hidden layers, 2D output) has many more parameters than the embedding table and can represent any mapping from symbol index to constellation point. But in practice the extra capacity is a disadvantage: at high SNR, many different constellations work equally well as long as the points are sufficiently separated, and the MLP tends to converge to arbitrary, scattered geometries that change with each run.

The embedding table, with only $2M$ parameters, forces each symbol to have exactly one fixed point. The initialisation on a QAM grid provides a good starting point, and the optimiser only needs to make small adjustments. The result is a constellation that is easy to interpret, easy to reproduce, and easy to export to a real transmitter (it is just a lookup table).

C. Practical considerations

From an embedded-systems perspective, the autoencoder has two attractive properties: it is small (about 19K parameters for 16-QAM) and it is fast (0.26 μs per symbol on a single CPU core). The decoder is a three-layer MLP that can be implemented with fixed-point arithmetic on a microcontroller or in an FPGA. The encoder is a lookup table, which is even simpler.

The main practical limitation is that the system needs a differentiable model of the channel in order to train. In a real deployment, this means either characterising the hardware in advance (measuring the AM/AM and AM/PM curves, the phase noise power spectral density) and building a model from measurements, or using a technique like reinforcement learning or generative adversarial networks to learn the channel model jointly. This is an active area of research [7].

Another limitation is that the system is trained for one operating point (one SNR, one set of impairment parameters). If the channel conditions change, the model needs to be retrained or fine-tuned. However, the curriculum training already exposes the system to a range of SNR values, which provides some robustness. And given that training takes less than 5 minutes on a laptop CPU, retraining on the fly is feasible for slowly varying channels.

D. Limitations

A few limitations should be noted:

- The channel model used in this work is memoryless (no inter-symbol interference). Extending it to frequency-selective channels or multi-carrier systems (OFDM) would require a higher-dimensional encoder and is left for future work.
- Gray coding is not enforced. The autoencoder minimises symbol error rate, not bit error rate. In practice the two are closely related for QAM, but a loss function that directly targets BER (e.g., by weighting bit differences) might yield further improvements.
- The system is single-antenna. MIMO extensions would increase the dimensionality of the problem but the same principles apply.
- We did not test on a real hardware testbed. The channel model is an approximation, and real impairments may differ in ways that affect the results.

IX. Conclusion

We have presented an autoencoder-based system for joint constellation design and symbol detection in QAM communication links affected by non-linear impairments. The encoder is a learnable embedding table that maps each symbol to a 2D point with power normalisation, and the decoder is a compact MLP. The two blocks are trained end-to-end through a differentiable channel model that includes AM/AM compression, AM/PM conversion, phase noise, and AWGN.

The main findings are:

1) Under AWGN only, the autoencoder recovers a near-standard QAM grid and matches the theoretical BER to within about 1 dB. This validates the approach on a well-understood baseline.
2) Under the full impaired channel, the autoencoder reduces the BER by up to $500\times$ compared to hard decision detection. Classical methods hit an error floor around 16% because the non-linearities destroy the regular grid geometry. The autoencoder breaks through this floor by jointly shaping the constellation and learning non-linear decision boundaries.
3) A DNN-decoder-only baseline (fixed QAM grid, learned detector) cannot match the full autoencoder under impairments. This proves that constellation shaping is essential, not just better detection.
4) The system is robust: multi-seed experiments show tight confidence bands, and ablation studies confirm insensitivity to the learning rate, hidden dimension, and curriculum schedule.
5) The autoencoder decoder runs 10–$12\times$ faster than minimum-distance detection on CPU, making it suitable for real-time embedded applications.

Future work could extend the approach to frequency-selective channels (OFDM), multi-antenna systems (MIMO), and hardware-in-the-loop training where the channel model is learned from measurements rather than specified in advance.

References

[1] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. McGraw-Hill, 2008.

[2] M. Katz and S. Shamai (Shitz), "On the outage probability of a multiple-input single-output communication link," *IEEE Transactions on Wireless Communications*, vol. 5, no. 1, pp. 5–14, 2006.

[3] T. O'Shea and J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Transactions on Cognitive Communications and Networking*, vol. 3, no. 4, pp. 563–575, 2017.

[4] S. Dorner, S. Cammerer, J. Hoydis, and S. ten Brink, "Deep learning based communication over the air," in *IEEE Journal of Selected Topics in Signal Processing*, vol. 12, no. 1, 2018, pp. 132–143.

[5] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proceedings of the 26th International Conference on Machine Learning (ICML)*, 2009, pp. 41–48.

[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

[7] F. A. Aoudia and J. Hoydis, "Model-free training of end-to-end communication systems," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 11, pp. 2436–2446, 2019.